VFIDE White Paper

Veritas + Fides — Truth + Trust

A self-custodial payments and commerce protocol with on-chain reputation pricing.

Version 1.0 · Codebase baseline v19.13 · Networks: Base Sepolia (testnet); Base, Polygon, zkSync Era (mainnet targets)

Table of Contents

1. Why VFIDE Exists
2. Design Principles
3. The Protocol in Plain English
4. Architecture
5. Economics: The Fee Curve and the Treasury
6. Howey Compliance Posture
7. Governance and the Path to Decentralization
8. Threat Model and Security Properties
9. Roadmap and Status
10. References and Further Reading

> ***Reading guide.*** *Sections 1–3 are written for any reader. Section 4 introduces the contract architecture. Section 5 covers economics in concrete terms. Section 6 is the legal/regulatory posture. Section 7 covers governance. Section 8 is the threat model. Sections 4–8 each include a short "in plain English" lead followed by the detail. The deepest level of detail — every constant, every function signature, every line citation — lives in the VFIDE Complete Manual (v1.0, Testnet Edition).*

---

1. Why VFIDE Exists

Roughly 1.4 billion adults have no bank account. Two to three billion more do have access to financial services but pay disproportionately for them — the small business owner who watches three percent of every card sale flow to networks she has never met; the freelance designer whose international client pays through a processor that takes four percent and a foreign-exchange spread that takes another two; the construction worker sending three hundred dollars home each month and watching a remittance company take eight to fifteen percent; the mobile-money user paying a flat fee on every send and every withdrawal that, on small balances, becomes a regressive tax on simply having money in motion.

These users are not unbanked in the technical sense. They have accounts. They are *extracted from*. The fees do not decrease with loyalty, volume, or proven honesty. There is no mechanism in the existing rails to reward a track record. A merchant who has processed ten thousand clean transactions pays the same percentage as one who opened an account this morning.

Crypto promised an alternative and largely delivered speculation. Wallets that require seed-phrase custody offer freedom to people who already have technical capacity and savings to lose; they offer a *worse* user experience than a bank to people who cannot afford to lose anything. "Banking the unbanked" became a pitch deck, not a product.

VFIDE is the attempt to take the actual primitives that distributed systems make possible — programmable settlement, transparent state, behavior that earns better terms — and aim them at the boring, common case: a person paying a person, a customer paying a merchant, a worker sending money home. The novelty is not in the cryptography. It is in *what the cryptography is used for*.

---

2. Design Principles

VFIDE is built on three commitments. Each is encoded in code, not in policy.

1. The merchant receives every cent of the listed price. A buyer who pays a merchant a posted price of 100 VFIDE causes the merchant's vault to receive 100 VFIDE. The protocol fee on merchant-of-record commerce transactions is hardcoded to zero (`MerchantPortal`, `protocolFeeBps = 0`). The buyer pays the network fee separately, on top, as a visible line item — the way a buyer pays sales tax in the United States, not the way a card swipe hides processing costs inside the price.

2. Trust is earned by behavior, not granted by authority — and earned trust lowers the cost of using the system. Every user has a **ProofScore** (0–10,000) computed on-chain from their own actions: completed transactions, endorsements from already-trusted users, governance participation, mentor relationships, dispute outcomes. The ProofScore is not transferable, not for sale, and not delegable. A new user pays the maximum fee. As the user accumulates a track record, the fee falls along a fixed, immutable curve. Trust replaces credit history.

3. No entity can freeze, blacklist, or seize another user's tokens after the bootstrap period. Not the developer. Not the elected council. Not any single user. The capability is *structurally absent*: the post-handover admin contract (`OwnerControlPanel`, 1,326 lines) has no `setFrozen` function, no blacklist, no seizure path. This is not a feature-flag in the off position. It is missing code. A regulator acting through the protocol cannot freeze tokens because the protocol does not have hands.

A fourth, derived principle follows from the first three: **the protocol team cannot break these guarantees by choice**. Six months after launch, the `SystemHandover` contract permanently transfers admin authority to `address(0)`. From that moment forward, the developer's keys are equivalent to any other user's keys. The bootstrap-period emergency-pause capability — limited even during bootstrap to *pausing*, never *freezing* — disappears with the key.

---

3. The Protocol in Plain English

A user opens the VFIDE app on their phone and connects a wallet. (The wallet can be MetaMask Mobile, Rainbow, Coinbase Wallet, or any other standard Web3 wallet. There is no signup form, no email, no KYC.) On first use, the app deploys a per-user smart-contract vault — a `CardBoundVault` — that will hold the user's VFIDE tokens. The phone holds a key. The vault holds the funds. **Lose the phone, the funds are still in the vault.**

Once the vault is set up, the user adds at least two **guardians** — other VFIDE users they trust personally, identified by VFIDE address. (A spouse, a sibling, a community leader, a trusted employer, a co-op treasurer.) Guardians can collectively authorize the user to rotate to a new wallet key if the old one is lost or stolen. Guardians cannot move

funds on the user's behalf. They can only attest, on a 14-day window with a 7-day challenge period, that the user is who they say they are when transferring control to a new key.

To send a payment, the user enters a recipient vault address and an amount, signs an EIP-712 typed `TransferIntent` with their wallet, and submits. The transaction settles on-chain in seconds. The recipient receives the full amount the sender entered. The sender additionally pays a network fee — between 0.25% and 5% depending on their ProofScore, capped at 1% on transfers of 10 VFIDE or less.

To accept payments as a merchant, a vendor enables Merchant Mode in the app, displays a QR code with their vault address, and scans the buyer's confirmation. The merchant receives 100% of the listed price. The buyer pays the network fee. The merchant pays nothing to the protocol.

A user's ProofScore goes up automatically as they transact, get endorsed by other trusted users, vote in governance, or mentor newcomers. It can go down through dispute losses, fraud findings, or extended inactivity (decay). The **rate of change is rate-limited and audited**: the `Seer` contract caps any single score adjustment at 1% of the score range per call, caps DAO-initiated adjustments at 5% per call with a 4-hour cooldown, and stores a 50-entry history per user that anyone can audit. **A user's score cannot be manipulated up before a large transfer**: the fee uses a 7-day time-weighted average of the score, not the current value.

Everything described in this section is implemented in code on testnet today.

---

4. Architecture

In plain English. VFIDE is a stack of single-purpose smart contracts that talk to each other through narrow, audited interfaces. Each user has their own vault. Reputation is computed by one contract. Fees are computed by another. The token contract orchestrates transfers and asks the fee contract what to do. Governance is a separate contract again. The split is intentional: each piece is small enough to audit; each upgrade is scoped to one piece.

4.1 Core contracts

The protocol comprises **108 Solidity contract files** at the v19.13 baseline. The five most important to understand are:

#	Contract	Lines of Code	Responsibility
1	`VFIDEToken`	1,454	ERC-20 with custom transfer path. Reads the `BurnRouter` result, validates returned sink addresses against its own configuration, executes the burn / Sanctum / Ecosystem split, applies anti-whale limits, routes to vaults. The hot path on every transfer.
2	`CardBoundVault`	1,536	Per-user vault. Holds tokens. Validates EIP-712 `TransferIntent` signatures with `walletEpoch` replay protection. Manages the guardian set, the 7-day withdrawal queue, and wallet rotation.
3	`Seer`	(large)	The ProofScore engine. Composition of score signals, time-weighted averaging, score history, decay, rate-limited adjustments, dispute resolution.
4	`ProofScoreBurnRouter`	1,011	Stateless fee computation. Linear interpolation curve, micro-transaction ceiling, sustainability redirects, volume-adaptive multiplier, daily burn cap.
5	`DAO`	(large)	Governance. Proposal types, voting period, fatigue, score-snapshot rules, queue and timelock pipeline.

A typical payment transaction touches all five: the wallet signs an intent → the `CardBoundVault` validates and instructs the `VFIDEToken` → the token consults `ProofScoreBurnRouter` for the fee, which consults `Seer` for the time-weighted score → the token executes the burn, the Sanctum transfer, the Ecosystem transfer, and the net delivery to the recipient vault.

4.2 Per-user vaults (`CardBoundVault`)

Each user has their own vault contract, deployed deterministically with `CREATE2` so the address is predictable before deployment. This isolates custody: if a bug in a vault somehow allowed an exploit (none has been found), the blast radius is one user, not all users. Vaults are issued and tracked through `CardBoundVaultDeployer` and (optionally) `VaultRegistry`.

Every state-changing call into a vault must carry a properly-signed EIP-712 `TransferIntent`. The intent commits the signature to the exact (`vault`, `toVault`, `amount`, `nonce`, `walletEpoch`, `deadline`, `chainId`) tuple. Each field defends against a specific class of attack:

- `walletEpoch` increments on every wallet rotation, invalidating all old signed intents the moment a recovery completes.
- `nonce` blocks replay within an epoch.
- `deadline` blocks indefinite-replay attacks against a still-valid epoch.
- `chainId` blocks cross-chain replay.

Withdrawals to external addresses are queued for **7 days** (`WITHDRAWAL_DELAY = 7 days`). Any of the user's guardians can cancel a queued withdrawal during that window. A user with a stolen phone has a week to revert any large drain.

4.3 ProofScore (`Seer`)

ProofScore composes from four families of signal:

- **Transactional** — completed transfers, especially repeat counterparties.
- **Social** — endorsements from already-trusted users (`SeerSocial`, `minScoreToEndorse`

`= 7,000`); mentor relationships (`minScoreToMentor = 7,200`).

- **Governance** — voting in DAO proposals, council membership, dispute participation.
- **Adverse** — fraud findings, dispute losses, extended inactivity (decay).

Three properties matter for the integrity of the system:

1. **Rate limits.** No single source can move a score by more than 1% of the range per call (`maxSingleReward = 100`). DAO-initiated adjustments are capped at 5% per call with a 4-hour cooldown (`maxDAOScoreChange = 500`, `DAO_SCORE_COOLDOWN = 4 hours`). This was tightened in audit-fix H-5.
2. **Time-weighted reads for fee computation.** The fee uses `getTimeWeightedScore(user)` over the last 7 days, walked from the user's 50-entry circular history. Manipulating the score upward right before a large transfer does not move the fee.
3. **Non-transferability.** ProofScore is bound to the user's address. It is not for sale, not delegable, not buyable. There is no on-chain action that transfers a score from one address to another.

4.4 Recovery (`Guardians`, `VaultRecoveryClaim`)

Guardians are addresses chosen by the user. The system enforces a maximum of **20 guardians** per vault (`MAX_GUARDIANS`). Guardian additions and removals route through a 7-day administrative delay (`SENSITIVE_ADMIN_DELAY = 7 days`), so a freshly compromised key cannot replace the guardian set in time to drain the vault.

The **happy path** for recovery is that the user has lost their wallet but still has their guardians: the guardians collectively call `approveWalletRotation` until the threshold is met, then the new key takes over and the `walletEpoch` increments, invalidating every still-pending signed intent on the old key.

The **unhappy path** is `VaultRecoveryClaim` — the nuclear option, used when the user has lost contact with their guardians too. A claim opens a **14-day guardian voting window** (`GUARDIAN_VOTE_WINDOW`) followed by a **7-day challenge period** (`CHALLENGE_PERIOD`). If the claim is uncontested at the end, ownership transfers to the new wallet. The latency is intentional: if it could be rushed, it would be the attack vector.

4.5 Other key components

- `Sanctum` — a community-funded buyer-protection pool that automatically receives 10% of every transfer fee. Used for dispute resolution payouts and fraud-victim restitution. Not a yield product. Sanctum holds funds; it does not pay them out as returns.
- `EcosystemVault` — receives the 50% Ecosystem share of each fee and redistributes it via `FeeDistributor` (see Section 5.2).
- `MerchantPortal` — the merchant-mode contract surface. `protocolFeeBps = 0`, meaning no protocol fee is taken from merchants on commerce transactions.
- `SystemHandover` — the 6-month developer-key burn timer. Single 60-day extension allowed if mainnet readiness is materially blocked (`monthsDelay = 180 days`, `extensionSpan = 60 days`).
- `OwnerControlPanel` — the post-handover admin contract surface. **Deliberately omits** `setFrozen` — there is no freeze function, not because it was removed, but because it was never written into this contract.

---

5. Economics: The Fee Curve and the Treasury

In plain English. Every transfer pays a small fee. The size of the fee depends on how trusted the sender is — a brand-new sender pays five percent, a long-trusted sender pays a quarter of one percent. The fee splits three ways: 40% is destroyed (which keeps the supply from growing), 10% goes to a community-funded buyer-protection pool, and 50% pays for running the protocol. On commerce, the merchant pays zero — only the buyer pays.

5.1 The fee curve

The fee on a vault-to-vault transfer is a piecewise-linear function of the sender's 7-day time-weighted ProofScore:

ProofScore (time-weighted)	Total fee
0 – 4,000 (`LOW_SCORE_THRESHOLD`)	**5.00%** (flat ceiling, `maxTotalBps = 500`)
4,000 – 8,000	linear interpolation 5.00% → 0.25%
8,000 – 10,000 (`HIGH_SCORE_THRESHOLD`)	**0.25%** (flat floor, `minTotalBps = 25`)

A **micro-transaction ceiling** of 1.00% applies to any transfer of ≤ 10 VFIDE (`microTxFeeCeilingBps = 100` , `microTxMaxAmount = 10 VFIDE`). The ceiling only activates if it *saves the user money* — a high-trust user with a 0.25% earned rate continues to pay 0.25% on micro-transactions. Trust is never penalized.

The score used is `getTimeWeightedScore` **over 7 days**, not the live score, so a user who briefly inflates their score before a large transfer pays the fee that matches the average of their actual recent behavior.

The curve is implemented in `_calculateLinearFee` at `ProofScoreBurnRouter.sol:494-512` . The constants are in `ProofScoreBurnRouter.sol:94-97` and `ScoringConstants.sol:18` .

For commerce, the path is different. Transactions routed through `MerchantPortal` carry `protocolFeeBps = 0` . The merchant receives the listed price. The buyer's transfer pays the buyer-side fee under the curve above, just as if they had paid any other VFIDE address. The merchant pays nothing to the protocol, ever, regardless of volume or ProofScore.

5.2 The treasury split

Whatever the total fee on a non-commerce transfer comes out to (5%, 0.25%, 1%, anything in between), it is split at the moment of transfer the same way:

- **40% burned.** Sent to the burn sink. Permanently removed from circulating supply.
- **10% to Sanctum.** The community-funded buyer-protection pool. Used for dispute resolution payouts and fraud-victim restitution.
- **50% to Ecosystem.** Sent to `EcosystemVault` , then redistributed by `FeeDistributor` across five sub-channels:
- **35%** to operations (infrastructure, RPC, monitoring, security retainers)
- **20%** to Sanctum secondary funding
- **15%** to council payroll (employment compensation for governance work; auto-swapped to stablecoins)
- **20%** to merchant rewards (verified-work payouts to merchants meeting verification criteria)
- **10%** to "headhunter" rewards (verified-work payouts for finding and onboarding new users)

(The percentages in the Ecosystem sub-split are percentages *of the 50% Ecosystem share*, configured in `FeeDistributor.sol:167-171` with immutable bounds `MIN_BURN_BPS = 2,000` and `MAX_SINGLE_BPS = 5,000`.)

A **sustainability floor** ensures that even at the high-trust 0.25% minimum, an absolute floor of 0.05% always reaches the Ecosystem fund. If circulating supply approaches `minimumSupplyFloor` (50,000,000 VFIDE), or if the daily burn cap (`dailyBurnCap = 500,000 VFIDE`) is reached, scheduled burns redirect to Ecosystem. The protocol cannot burn itself out.

5.3 Token supply

Constant	Value	Source
`MAX_SUPPLY`	200,000,000 VFIDE	`VFIDEToken.sol:52`
`DEV_RESERVE_SUPPLY`	50,000,000 VFIDE	`VFIDEToken.sol:53`
Genesis treasury allocation	150,000,000 VFIDE	`VFIDEToken.sol:298`
Dev vesting cliff	60 days	`DevReserveVestingVault.sol:39`
Dev vesting period	60 months (1,800 days)	`DevReserveVestingVault.sol:40`
Dev vesting unlock interval	60 days (bi-monthly)	`DevReserveVestingVault.sol:41`
Dev vesting unlock amount	1,666,666 VFIDE	`DevReserveVestingVault.sol:42`
Dev vesting total unlocks	30	`DevReserveVestingVault.sol:43`

There is no presale. The token launches via a **Liquidity Bootstrapping Pool (LBP)**, a Balancer-style mechanism in which the price starts high and decreases over a 48–72-hour window, allowing the market to discover the price without privileged early-buyer pricing tiers. The original three-tier presale ($0.03 / $0.05 / $0.07) was permanently removed; the corresponding 75M presale allocation was redistributed (50M to dev reserve, 150M to system allocation). Lock bonuses and referral bonuses in the original presale contract were permanently disabled.

There is no staking. There is no APY. There is no yield. There is no `LiquidityIncentives` reward. The `LiquidityIncentives.sol` contract (270 LOC) exists only to coordinate liquidity at the protocol level; it explicitly pays liquidity providers nothing. The `DutyDistributor.sol` contract tracks governance participation as non-monetary "Duty Points" — badges of participation, not value.

For the legal-posture rationale behind those structural choices, see Section 6.

5.4 Anti-whale limits

The token contract enforces four configurable limits (`VFIDEToken.sol:59-62`). Each can be disabled by DAO vote setting it to zero. Exchanges, liquidity pools, and protocol contracts are exempt:

Limit	Default	Purpose
`maxTransferAmount`	2,000,000 VFIDE (1% of supply)	Caps any single transfer
`maxWalletBalance`	4,000,000 VFIDE (2% of supply)	Caps any single wallet/vault balance
`dailyTransferLimit`	5,000,000 VFIDE (2.5% of supply)	Rolling 24-hour outflow per sender
`transferCooldown`	0 seconds (disabled)	Minimum time between transfers

---

6. Howey Compliance Posture

This section documents how the protocol is *structurally designed* to fail each of the four prongs of the Howey test, what design choices were made to weaken existing prongs, and where the residual exposure lies.

This section is not legal advice. The question of whether the VFIDE token is a security under U.S. law (or any other jurisdiction's law) is a question for qualified counsel, and one that ultimately depends on facts about marketing, sale practices, and conduct that no codebase can fully control. What this section can document is the **design posture**: what the contracts do and do not do, and how those choices map to each Howey prong.

Prong 1 — Investment of money

Status: Fails. There is no presale. The original three-tier presale ($0.03 / $0.05 / $0.07) was permanently removed; the 75M presale allocation was redistributed (50M to dev reserve, 150M to system allocation). Tokens are acquired one of three ways:

- **LBP launch** — price discovered by the market, no privileged early-buyer tier.
- **Faucet, referrals, work rewards** — earned through usage, not bought.
- **Open-market secondary trading** — once the LBP completes.

Lock bonuses and referral bonuses in the original presale contract were permanently disabled.

Prong 2 — Common enterprise

Status: Weakened. Some horizontal commonality is unavoidable in any token system: the deflationary burn implicitly affects every holder (reduced supply), and `EcosystemVault` pools fees collectively. The mitigations are structural:

- **Fee reduction is framed and implemented as a utility-cost discount**, not a return on capital. A user with a high ProofScore pays *less* on their own transactions; the system does not pay *them* anything.
- **Council payments are framed as employment compensation.** The `CouncilSalary` contract literally hardcodes the string "EMPLOYMENT COMPENSATION, not investment returns." Salary is auto-swapped to stablecoins (USDC) to avoid treating governance pay as native-token appreciation.
- **Work rewards are paid for verified work**, not for token-holding. Events emit `WorkRewardPaid` with proof attestations showing the work performed.
- **Headhunter rewards** require verified user onboarding actions, not capital deployment.

Prong 3 — Expectation of profits

Status: Weakened. The deflationary burn creates an implicit scarcity narrative that no token system can fully avoid. Mitigations:

- The metric `totalBurnedToDate()` was renamed `transactionFeesProcessed()` to frame the metric as **network cost**, not value accrual.
- ProofScore fee reduction is framed as a utility discount (cheaper transactions when you're trustworthy), not as an investment return.
- **No staking. No yield. No APY anywhere in the protocol.**
- `LiquidityIncentives.sol` (270 LOC) explicitly provides **zero rewards**. The contract exists to coordinate liquidity but does not pay liquidity providers.
- `DutyDistributor.sol` (133 LOC) tracks governance participation as non-monetary "Duty Points" — badges of participation, not value.
- Public-facing copy is audited for prohibited terminology: no "yield," "stake," "APY,"

"passive income," "investor returns," or "rewards-as-returns" appears anywhere in the user interface, marketing site, or documentation.

Prong 4 — Efforts of others

Status: Fails. After `SystemHandover` (6 months post-launch, 60-day extension allowed), the developer has **no privileged access**. The administrative key is set to `address(0)`. There is no path back. ProofScore is user-controlled — earned through individual behavior, not team effort. Fee reduction comes from personal trust-building, not from anything the protocol team does. There is no centralized management post-handover.

`OwnerControlPanel.sol:1032` hardcodes `howey_areAllSafe()` to return `true` unconditionally. **Howey-safe mode is permanently hardcoded across every ecosystem contract** — there is no toggle to flip; the safe mode IS the contract.

Residual exposure

The protocol cannot, by design alone, prevent every possible interpretation. Specifically:

- **Marketing and sale conduct** can override structural design. If a third party promotes VFIDE as an investment, that conduct creates exposure no smart contract can prevent. The protocol team commits to and enforces (through governance) marketing language audits aligned with this section.
- **Bootstrap-period developer privileges** are limited to *pause*, never *freeze*, and end at handover. During that window the protocol does rely on the developer multisig acting in good faith.
- **Token appreciation in secondary markets** is outside the protocol's control. The protocol does not promise it, structure for it, or distribute returns from it.

---

7. Governance and the Path to Decentralization

In plain English. VFIDE is governed by an elected council of twelve people. Their votes are weighted by their ProofScore — the same earned-trust number that determines users' fees. So a person who has been honest in the system for years has more voting power than a person who just bought a million tokens this morning. After six months, the developer's admin key permanently burns and the protocol becomes fully community-governed.

7.1 Reputation-weighted voting

The DAO uses **ProofScore as voting weight, not token balance.** A user holding ten million VFIDE with a ProofScore of zero has zero voting power. A market vendor with two years of clean transaction history has substantial voting power. This is the central anti-capture defense: token-buying does not buy governance.

Constant	Value	Source
`MIN_GOVERNANCE` (eligible to vote)	ProofScore ≥ 5,400	`ScoringConstants.sol:32-35`
`MIN_MERCHANT`	ProofScore ≥ 5,600	`ScoringConstants.sol:32-35`
`COUNCIL_MIN_SCORE` (eligible for council)	ProofScore ≥ 7,000 (immutable)	`CouncilManager.sol:56`
`councilSize` (default)	12 (configurable 1–21)	`CouncilElection.sol:97`
`FIXED_TERM_SECONDS` (term length)	365 days (immutable)	`CouncilElection.sol:25`
`FIXED_MAX_CONSECUTIVE_TERMS`	1 (immutable, no consecutive terms)	`CouncilElection.sol:24`
`votingPeriod` / `votingDelay`	7 days / 1 day	`DAO.sol:86,89`
Council action timelock	48 hours (default), 24 hours (floor)	`DAOTimelock.sol:43,107`
`EMERGENCY_RESCUE_DELAY`	14 days	`DAO.sol:97`
`FATIGUE_PER_VOTE` / `RECOVERY_RATE`	5% / 5% per day	`DAO.sol:163-164`

Council members must maintain `ProofScore ≥ 7,000` to remain seated; auto-removal triggers after 7 days below threshold. Fixed one-year non-consecutive terms prevent entrenchment. Every council action passes through a 48-hour timelock during which any community member with score ≥ 5,000 can propose veto via `AdminMultiSig`.

7.2 Vote fatigue

A novel mechanism: each cast vote incurs **5% voting fatigue**, recovering at **5% per day**. A council member who tries to vote on every proposal in a single day approaches zero effective voting power; a council member who picks their battles retains full weight. This pushes council attention toward the proposals that actually matter, and it makes governance attacks (spamming proposals to exhaust an honest council) self-defeating.

7.3 The handover

The `SystemHandover` contract holds a single function: a 180-day timer (single 60-day extension allowed) that, when fired, sets the developer admin address to `address(0)`. After the call, no path back exists. The post-handover admin contract — `OwnerControlPanel.sol`, 1,326 lines — is deliberately a smaller surface than any pre-handover admin contract: it can adjust parameters within scope, it can pause emergencies briefly (with timelock), and it has no `setFrozen`, no blacklist, no seizure path. Those functions are not in the file. They were never written into this contract.

This is the moment at which Prong 4 of Howey fails by structural design rather than by promise.

---

8. Threat Model and Security Properties

In plain English. A user with a stolen phone has a week to undo any large drain via guardians. A user who loses contact with everyone can recover through a slow, public claim process. A compromised oracle, a compromised council, or a compromised single contract cannot freeze the user's funds — the freeze function does not exist. A bug in one user's vault affects only that user.

8.1 What VFIDE defends against

Threat	Defense
Phone theft + immediate drain	Daily and per-transfer limits cap extraction per window. Withdrawals to external addresses queue 7 days; any guardian can cancel during that window.
Phone theft + slow drain	Guardians notice unusual activity, can pause the vault for 7 days while user rotates wallet.
Compromised wallet key with full custody	`walletEpoch` invalidates all old signed intents at rotation; user signs new intents with new key.
Lost phone + lost guardians	`VaultRecoveryClaim` — 14-day guardian voting window + 7-day challenge period.
Replay attacks	EIP-712 typed data binds signature to (`vault` , `toVault` , `amount` , `nonce` , `walletEpoch` , `deadline` , `chainId`). Each field defends a specific replay variant.
Score manipulation before large transfer	Fee uses 7-day time-weighted score, not live score.
Single bad actor inflating one user's score	`maxSingleReward = 100` (1% per call). DAO adjustments capped at 5% per call with 4-hour cooldown.
Compromised `BurnRouter`	Token re-validates returned sink addresses against its own configured `treasurySink` , `sanctumSink` , `burnSink` (audit fix F-17/C-01). Worst case: revert.
Governance capture by token whale	Voting weight is ProofScore, not token balance. Whale with no track record has no voting power.
Governance capture by entrenched council	1-year non-consecutive terms; auto-removal below score 7,000; 48-hour timelock; community veto via AdminMultiSig.
Malicious upgrade or freeze via admin	`setFrozen` does not exist in `OwnerControlPanel` . After `SystemHandover` , admin key is `address(0)` .
Bug in one user's vault	Per-user `CardBoundVault` deployed via CREATE2 — blast radius is one user.

8.2 What VFIDE does not defend against

Honest documentation requires listing what the protocol does *not* protect against:

- **Operational mistakes by the user.** Sending to a wrong address that the user has personally entered. Approving a phishing site. Sharing a private key.
- **Marketing conduct by third parties** that could create legal exposure regardless of contract design (see Section 6, residual exposure).
- **Failures of the underlying L2.** A catastrophic failure of Base or its sequencer. The protocol can be redeployed on alternate networks but cannot prevent base-layer failures during incidents.
- **Bridge risk** during cross-chain transfers. `VFIDEBridge` enforces a 7-day refund delay (`BRIDGE_REFUND_DELAY = 7 days`) and a default 0.1% bridge fee but, like any bridge, depends on the underlying chains and validator sets it connects.
- **Bootstrap-period developer trust.** The first 6 months trust the developer multisig to handle emergencies responsibly via *pause*, never *freeze*. After `SystemHandover`

this assumption disappears entirely.

8.3 Audit posture

The internal audit catalog tracks **66+ findings across multiple cycles**. Every threshold in this document and in the Complete Manual matches the post-fix code at the v19.13 baseline. **Public audit publication is planned ahead of mainnet deployment.** The test surface comprises 473 test files (Solidity behavioral, frontend unit, API integration, E2E user journeys, performance) totaling over 9,000 individual tests, all passing on the current baseline. A continuous integration pipeline runs the full suite on every change.

---

9. Roadmap and Status

Phase	Contracts	Status
Phase 1 — Core	Token, vault, score, governance, lending, fraud, faucet (18 contracts)	**Live on Base Sepolia testnet**
Phase 2 — Recovery + Council	`VaultRecoveryClaim`, `CouncilElection`, `CouncilManager`, `SystemHandover`	**Live on Base Sepolia testnet**
Phase 3 — Commerce	Beyond `MerchantPortal`: catalog, escrow, dispute layer	In active development
Phase 4 — Achievements	Badges, reputation NFTs (non-financial)	Planned
Phase 5 — Autonomous Seer	Reduced-trust ProofScore mutation paths	Planned
Phase 6 — Bridge	`VFIDEBridge` to Polygon, zkSync Era	Contract written; activation post-mainnet

Item	Status
Solidity source	108 contract files at v19.13 baseline
Test coverage	473 test files, 9,000+ tests passing
Audit findings tracked	66+, all addressed in current baseline
Mainnet target	Base (primary), Polygon, zkSync Era
Developer key burn	6 months post-mainnet via `SystemHandover`
Public audit publication	Planned ahead of mainnet

---

10. References and Further Reading

- **Executive summary (1 page):** `WHITEPAPER-EXECUTIVE-SUMMARY.md`
- **VFIDE Complete Manual (v1.0, Testnet Edition):** the canonical reference. Every threshold cited in this paper appears there with the exact source-code line citation. Any disagreement between this paper and the code: the code wins. The manual contains the full Quick Start (Adwoa's first ten minutes), the User Manual (10 chapters), the Technical Reference (T1–T7, including T6 Howey Compliance Notes and T7 Test Coverage Map), and the Appendices (130-entry glossary, FAQ, troubleshooting, Reference Card).
- **Source code:** `github.com/Scorpio861104/Vfide`

- **Security disclosure policy:** `SECURITY.md`
- **Sustainability model:** `docs/SUSTAINABILITY-MODEL.md`
- **Technical reference (frontend / API surface):** `docs/VFIDE-TECHNICAL-REFERENCE.md`

Disclaimer. This document describes the design and current status of the VFIDE protocol. It is not an offer to sell securities, not investment advice, and not legal advice. Token availability and protocol features are subject to applicable law in each jurisdiction. The Howey-prong analysis in Section 6 reflects the protocol's design posture and is not a legal opinion. Consult qualified counsel before participating in any token launch.

Veritas + Fides — Truth + Trust.